Loop Engineering and Harness Engineering: A Comparative Analysis of Two Disciplines for Operating Autonomous Coding Agents

Duy Tran Thanh
duy.tran10@onemount.com

Abstract

The rapid capability gains of large language model (LLM) coding agents have shifted the locus of engineering effort away from the model and toward the system that operates it. Two named disciplines emerged in 2026: loop engineering*, the design of the self-running cycle that prompts an agent toward a goal, and* harness engineering*, the design of the full scaffolding that surrounds a model to make it reliable. The two are frequently conflated. This report formalizes both, argues that they stand in a part–whole relation rather than competing, and characterizes the distinct failure modes each addresses. We give a six-tuple formal model of a loop, identify two safety invariants (mandatory verification, mandatory termination) that distinguish an engineered loop from the colloquial "Ralph loop," and ground the analysis in OnLOOP, a contracts-first reference implementation. We conclude that loop engineering is the temporal/control-flow specialization of harness engineering, and that the chief practical contribution of treating the loop as a first-class object is the promotion of verification and termination from discretionary practices to type-level obligations.*

1. Introduction

A coding agent is increasingly described by the decomposition $\text{Agent} = \text{Model} + \text{Harness}$, where the *model* supplies intelligence and the *harness* is "everything in an AI agent except the model itself" [3, 2]. As frontier models saturate single-prompt benchmarks, the marginal return on prompt-level optimization falls and the return on system-level design rises. Two disciplines name this system-level work; Fig. 1 previews how they differ.

Loop engineering is "building a system that prompts your agent on a schedule and against a goal, instead of typing each prompt yourself" [1]. Its thesis: leverage has moved from the quality of a single prompt to the design of the system that *generates and verifies* prompts.

Harness engineering is "the discipline of designing, building, and iterating on the scaffolding that surrounds a language model" [3]: context delivery, tool interfaces, guardrails, memory, sandboxes, and feedback loops.

Practitioners often use the terms interchangeably, or treat them as rival schools. We argue this is a category error and make four contributions: (1) a formal six-tuple model of an engineered loop and its execution semantics (Sec. 4); (2) a characterization of harness engineering as a feedforward/feedback control structure, with a proof-sketch that the loop is a proper subcomponent of the harness (Sec. 5); (3) a multi-dimensional comparison establishing that the disciplines optimize orthogonal axes (Sec. 6); and (4) an empirical grounding in OnLOOP, whose type system enforces two invariants that the bare Ralph loop leaves optional (Sec. 7).

2. Background and Related Work

2.1. The agent loop

The cyclic structure of a rational agent (perceive, deliberate, act, observe) predates LLMs by decades. It is the basic control structure of the Belief–Desire–Intention model and is codified in standard textbooks [8]. LLMs did not invent the loop; they made a general-purpose actor cheap enough to place inside it, which unlocked open-ended agentic behavior [7].

2.2. Loop engineering and the Ralph loop

The phrase *loop engineering* was popularized by Osmani [1] in June 2026, crystallizing observations by Steinberger ("you should be designing loops that prompt your agents") and Cherny ("my role shifted to write loops rather than to prompt the model directly"). Its minimal instantiation is the **Ralph loop** [4], whose canonical form is one line:

```
while :; do cat PROMPT.md | agent ; done
```

Its insight is that progress lives in files and git history, not in the context window: each iteration starts a fresh agent with a fixed prompt, does a little work, writes results to disk, and exits. The convention separates a fixed task (`PROMPT.md`), accumulated learnings (`AGENTS.md`), and a progress log (`progress.md`) [4, 5]. Critically, the bare Ralph loop

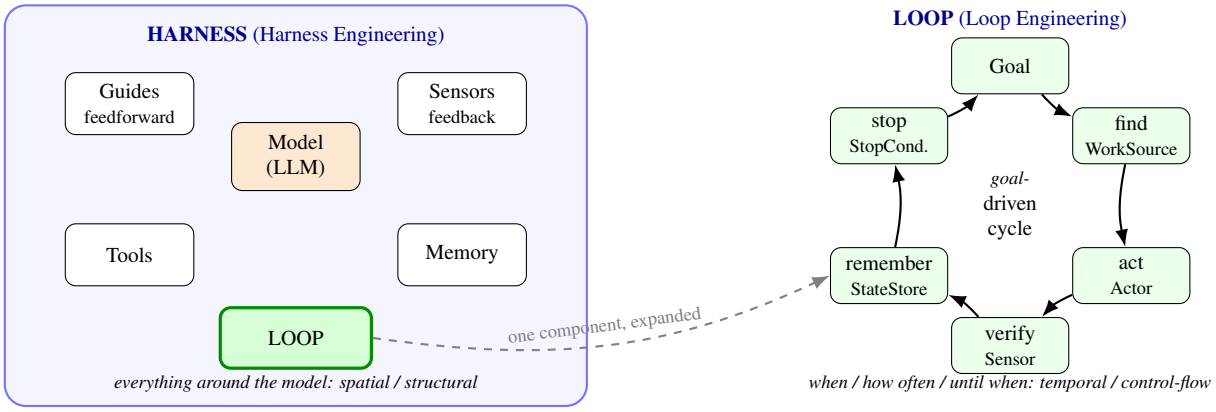

Figure 1: Overview: harness engineering vs. loop engineering. *Left:* the harness is everything around the model—guides (feedforward), sensors (feedback), tools, memory, and the loop—engineered along a *spatial* axis (what surrounds the model). *Right:* the loop is one component of the harness, shown expanded; loop engineering designs its *temporal* cycle Goal → find → act → verify → remember → stop, and owns termination. The two disciplines are a part and a whole, not rivals: harness engineering draws the box; loop engineering animates one part of it over time ($L \subsetneq H$).

has *no required verifier and no termination condition*; the `while :;` is literally infinite. We return to this in Sec. 7.

2.3. Harness engineering

Harness engineering was developed by Böckeler [3] at Thoughtworks and elaborated by Osmani [2] and LangChain [6]. It frames the harness as two classes of control: *feedforward* guides (conventions, documentation, templates, linters set ahead of time) and *feedback* sensors (tests, static analysis, review agents that observe output and enable self-correction). LangChain reports that an explicit four-stage structure (Plan, Build, Verify, Fix) outperforms unstructured loops [6], evidence that harness structure, not raw model capability, governs reliability.

3. Definitions

We fix terminology. The **model** (M) is the LLM, a black-box stochastic function from context to text/tool-calls. The **harness** (H) is the deterministic-and-inferential system around M; everything that is not M. The **loop** (L) is the control structure that repeatedly invokes the agent toward a goal; a component of H. A **sensor** observes an action's result and returns a verdict with evidence (feedback). A **guide** constrains generation before it occurs (feedforward).

4. A Formal Model of Loop Engineering

4.1. The loop as a six-tuple

We model an engineered loop as $L = (G, W, A, S, M_s, T)$, with components given in Table 1.

4.2. Inputs and outputs

Readers reasonably ask what loop engineering *consumes* and *produces*. Viewed as a function, a loop has the signature

$$\texttt{run} : \underbrace{(G, W, A, S, M_s, T)}_{\text{config}} \times \underbrace{(\mathsf{Wksp},\ M,\ \mathsf{Budget})}_{\text{environment}} \rightarrow \underbrace{(r,\ D,\ \sigma^*,\ \mathcal{H},\ \mathsf{Wksp}')}_{\text{outcome}}$$

summarized in Fig. 2. The **input** is, in one phrase, *a goal plus the means to pursue and check it*: the objective G with its success predicate, the model M as the actor's oracle, the workspace Wksp to act on, any state σ_0 restored from a prior run (so a loop resumes rather than restarts), and the verification/termination policy (S, T, budget). The **output** is *a terminated, audited outcome*: a stop reason r; a done-set D containing only items that passed their sensors (by P1); the mutated work product Wksp' on disk; a replayable per-iteration trace $\mathcal{H}$; and a final state σ^* that is itself a valid input to the next run. The output is thus *self-describing* (why it stopped), *verifiable* (what passed), and *resumable* (where to continue). This contrasts with harness engineering, whose I/O is per-*step*: it maps a model plus task context (guides, tools, memory) to one constrained, sensor-checked action. A loop's I/O is the closure of the harness's I/O over many steps until T fires.

4.3. Execution semantics

A run is generated by applying the transition rule of Fig. 4 from an initial state σ_0 loaded from M_s. The timestamp τ is supplied by the runtime, not read by any contract, which keeps a run replayable from its recorded trace.

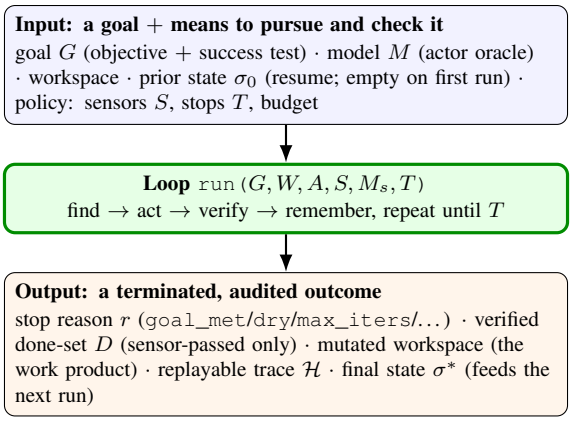

Figure 2: Inputs and outputs of a loop. Loop engineering turns a *goal* (plus a model oracle, a workspace, and a verification/termination policy) into a *terminated, verified outcome* whose final state is a valid input to the next run.

4.4. Properties

Two properties follow directly and are the heart of the discipline.

- **P1 (Verified progress).** By rules (R2)–(R3), an item enters the done-set only if the conjunction of all sensors passes. No sensor, no progress: an action the loop cannot check does not advance the goal.
- **P2 (Guaranteed termination).** If T contains at least one bound monotone in a quantity the runtime increments every pass (iterations, accumulated cost, or consecutive empty rounds), the run halts. Rule (R1) evaluates T before any work, so the bound cannot be skipped.

P1 and P2 are not automatic; they hold only if $S \neq \emptyset$ and T contains a real bound. An *engineered* loop is precisely one in which these are obligations rather than options (Sec. 7).

4.5. Patterns as instances of L

Common patterns are recovered by choosing components, not by writing new control flow: the *Ralph loop* fixes G to a spec, sets M_s to files+git, and leaves S, T empty (its weakness); *loop-until-dry* puts a k-empty-round bound in T; *fan-out/verify* lets W yield independent items under isolation; *adversarial verify* makes S a panel whose verdict requires a majority to survive refutation.

5. A Formal View of Harness Engineering

5.1. The harness as a control structure

Following Böckeler [3], the harness is a pair of control populations over the model, $H = (Guides, Sensors, Tools, Memory, L)$, where *Guides* are feedforward and *Sensors* are feedback. The central operating rule is a *meta-loop*: whenever an issue recurs, the feedforward and feedback controls are improved. This is a second-order loop in which the engineer (or, prospectively, an automated guide-writer) updates $Guides$ and $Sensors$ in response to recurring failures.

5.2. Inputs and outputs

The harness has an I/O contract too, but a per-*step* one. Treating a single agent step as a function,

$$\texttt{step}: \; M \times Ctx \; \rightarrow \; (a,\, v,\, \Delta\mu,\, \mathrm{tr}),$$

where the harness both *assembles* the input context Ctx (feedforward) and *checks* the resulting action (feedback); Fig. 3 expands it.

Input: the model M and the context Ctx. The model supplies capability; Ctx is everything the harness packs into the window for this step:

- *Task* — the current instruction or subtask.
- *Guides* (feedforward) — conventions, architectural rules, templates, linter configuration: constraints applied *before* generation.
- *Tools* — the action interface available this step (edits, shell, search, MCP servers).
- *Memory* — retrieved relevant state: prior decisions, accumulated learnings (`AGENTS.md`), code excerpts.
- *Sandbox* — the execution environment and its permissions.

Assembling this within the context window is itself the engineering work ("context engineering").

Output: a checked step $(a, v, \Delta\mu, \mathrm{tr})$.

- a — a single action (edit, tool call, message), possibly blocked or rewritten by a guardrail before it takes effect.
- v — the sensor verdict on a, *with evidence*: a conjunction of *computational* sensors (deterministic, fast: linters, tests, type-checkers) and *inferential* sensors (semantic, slower: review agents, LLM judges).
- $\Delta\mu$ — side effects: the mutated workspace if a is applied, and any learnings written back to memory.
- tr — telemetry for observability (the step's cost, decision, and verdict).

Harness engineering is precisely the design of *what enters Ctx* (feedforward) and *which sensors produce v* (feedback); it turns raw model capability into a reliable, *checked* step. The relationship to the loop is exact: the loop's per-run signature (`run`, Sec. 4, Fig. 2) is the closure of `step` iterated under the control flow of T, threading each $\Delta\mu$ into the next step's Ctx and aggregating each v into verified progress. Where harness I/O asks "is this one step good?", loop I/O asks "is the whole goal met, and when do we stop?".

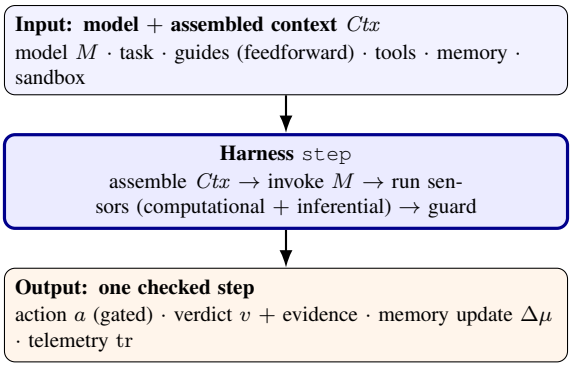

Figure 3: Inputs and outputs of a harness *step* (cf. the loop's per-run I/O, Fig. 2). Harness engineering designs the feedforward (what fills Ctx) and the feedback (which sensors yield v), turning model capability into one reliable, checked action. A loop iterates this step until T fires.

5.3. The loop is a subcomponent of the harness

Proposition. $L \subsetneq H$.

Argument. The harness is defined as everything that is not the model. The loop's components W, A, S, M_s, T are each non-model procedures, hence elements of H. The loop's sensors S are exactly the feedback-control population; its memory M_s is the harness memory; its actor A is the harness's invocation of the model. The loop additionally contributes control flow (T and the transition rule) that the static harness does not. Conversely H contains *Guides* and *Tools* outside the loop's defining tuple. Hence $L \subsetneq H$. □

This is the central structural claim: *loop engineering is the control-flow specialization of harness engineering.* They are not rivals; one is part of the other.

5.4. Two loops, often conflated

There are two distinct cycles: the *inner agent loop* (perceive–act –verify), a harness component formalized as L; and the *outer steering loop*, Böckeler's meta-loop, in which a human improves guides and sensors over time. Loop engineering, in its strong form, is partly the project of automating the outer loop (e.g., appending learnings to `AGENTS.md`) so supervision scales sublinearly with agent activity.

6. Comparative Analysis

We compare along ten dimensions, summarized in Table 2.

6.1. Object and axis (orthogonality)

The cleanest distinction is *axis*. Harness engineering operates on a *spatial* axis: breadth of scaffolding around the model at one instant. Loop engineering operates on a *temporal* axis: cadence and control flow across many invocations. One can improve the harness (a linter, a better template) without changing the loop, and change the loop (schedule it, add loop-until-dry) without changing the guides. Orthogonality is the operational signature that two disciplines are distinct rather than synonymous.

6.2. Failure mode (the diagnostic test)

Reach for *harness* vocabulary when the symptom is "the agent produces wrong or unconstrained output"; the fix is in guides and sensors. Reach for *loop* vocabulary when the symptom is "the agent is capable, but I am stuck driving every iteration"; the fix is a scheduled, goal-driven, self-verifying cycle.

6.3. Verification and termination

Both disciplines centre verification, which is why they are easy to conflate: they refer to the same artifacts (tests, linters, review agents) under different framings, a static taxonomy of controls versus a stage in a dynamic cycle. *Termination*, by contrast, is owned by loop engineering and largely ignored by harness engineering: a static harness has no notion of when to stop. Property P2 and the obligation $T \neq \emptyset$ only make sense once the loop is a first-class object.

7. Case Study: OnLOOP

OnLOOP is a contracts-first reference implementation: an existence proof that the formal model is realizable and that the safety invariants can be enforced mechanically.

7.1. Architecture

OnLOOP implements the six-tuple as six abstract interfaces (`Goal`, `WorkSource`, `Actor`, `Sensor`, `StateStore`, `StopCondition`). A concrete loop is one implementation of each, declared in a YAML spec that binds a contract to a class via a dotted path. A runtime executes the rule of Fig. 4: stop conditions are checked first (R1), progress requires all sensors to pass (R2–R3), and state is persisted every pass (R4) through a JSON `StateStore`, making runs restartable.

7.2. Invariants as type-level obligations

OnLOOP's spec schema enforces P1 and P2 *at load time*: `sensors` requires at least one entry (a loop that cannot check itself will not load, enforcing $S \neq \emptyset$), and `stop` requires at least one entry (a loop that cannot terminate will not load, enforcing $T \neq \emptyset$). This is the report's main practical claim: the value of treating the loop as a first-class object is the promotion of verification and termination from

optional practices to non-negotiable, machine-checked obligations.

7.3. The Ralph loop as a degenerate instance

In the model of Sec. 4, the bare Ralph loop is the instance with $S = \emptyset$ and $T = \emptyset$. Both safety properties fail. OnLOOP rejects exactly this configuration:

> *The Ralph loop is the loop with the brakes optional. OnLOOP is the same loop with the brakes required.*

7.4. Empirical sanity check

A deterministic, model-free reference loop (clearing a punch list on disk) exercises every component and each stop reason. Its end-to-end suite confirms: convergence to `goal_met` with verified progress; correct bounding by `max_iters` before goal; `loop_until_dry` termination on an empty work source; that a permanently failing sensor blocks an item from the done-set (a direct test of P1); and that state round-trips across a simulated restart (a test of R4). All cases pass. Because the actor is deterministic, the suite isolates the control structure from model stochasticity, which is the property under test. We make no claim about model-in-the-loop performance; that is future work.

8. Integrating a Loop into a Task

Loop engineering is applied by answering six questions about a task—one per contract—and binding each answer to an implementation. We give the recipe, then a worked example and the surfaces it plugs into.

8.1. Recipe: from a task to a loop

1. **State the goal G as a check, not a wish.** Name the command or predicate true exactly when the task is done ("`pytest` exits 0", "no `TODO` markers remain"). If you cannot write the check, the task is not yet loop-ready.
2. **Define the work source W.** Decide how an iteration discovers the next unit (parse failing tests, scan open items, pull a queue), skipping anything already in the done-set.
3. **Implement the actor A.** The smallest step that advances one unit—usually a single model call with a focused prompt, or a deterministic edit. Keep it small so a bad step is cheap to discard.
4. **Wire the sensor(s) S *before* the actor.** At least one verifier with evidence; prefer a fast computational sensor (tests, linter), add an inferential one (a review agent) only where semantics matter.
5. **Choose the state store M_s.** The default JSON store suffices; pick a *stable* done-key per work item so resumption never redoes work.
6. **Set stop conditions T.** Always `GoalMet` plus a real bound (`MaxIterations`, a token budget, or `LoopUntilDry`); never ship a loop without one.

Steps 1, 4, and 6 are the non-negotiable ones: they establish the preconditions of P1 and P2. Isolation and budget are optional policy layered on top.

8.2. Worked example: drive a test suite to green

The bindings above become a declarative spec (abbreviated):

```
name: make-tests-green
goal:        { uses: myloops.AllTestsPass,
               with: { cmd: "pytest -q" } }
work_source: { uses: myloops.FailingTests }
actor:       { uses: myloops.FixOneTest,
               with: { model: <llm> } }
sensors:
  - { uses: myloops.PytestSensor }
  - { uses: myloops.RuffSensor }   # 2nd check
stop:
  - { uses: onloop.stops.GoalMet }
  - { uses: onloop.stops.MaxIterations,
      with: { max_iters: 25 } }
budget:    { max_tokens: 400000 }
isolation: worktree
```

A runtime loads the spec and executes the rule of Fig. 4: each pass finds one failing test, the actor proposes a fix, *both* sensors must pass for it to count as done, and the run halts at `goal_met`, the iteration cap, or the token budget. Because state is persisted every pass, an interrupted run resumes where it stopped (`onloop resume`).

8.3. Where a loop plugs in

The same loop integrates into different surfaces by changing only *when* it runs and *what bounds* it: a *scheduled* job (run nightly until `dry`); a *pre-merge gate* (run once; fail the build unless `goal_met`); a *batch migration* (one work item per file, with `isolation: worktree` so parallel actors do not collide); or a *supervised* run (a human-on-the-loop checkpoint every k iterations). The contract bindings are unchanged; only T and the trigger differ.

8.4. Pitfalls

The recurring failures each violate a precondition of P1 or P2: an unverifiable goal (no check); the actor running before any sensor exists (fast, unchecked mistakes); unstable done-keys (work repeats on resume); and a missing real bound (an effectively infinite loop). The schema-level obligations of Sec. 7 catch the last two at load time.

9. Discussion

Fig. 5 gives the unifying picture: harness engineering draws the box; loop engineering animates one component of it over time. The disciplines are complementary and a mature system needs both. A reasonable order of operations: establish a harness with at least one feedback sensor and a useful guide; then engineer the loop that drives the agent through it, fixing termination and verification first because they are the cheap, high-consequence invariants. Conflating the two yields two characteristic mistakes: treating loop engineering as the whole story gives the Ralph failure (a slick autonomous cycle with no brakes); treating harness engineering as the whole story gives a well-instrumented agent that still needs a human in the chair for every step.

10. Limitations and Threats to Validity

Construct validity: both terms are recent (2026) and not standardized; our definitions follow the originating sources but usage is in flux. **Single implementation:** our grounding is one framework with a deterministic actor; we show the invariants are enforceable and the control structure correct, not that LLM-actor loops converge efficiently on real tasks. **No model-in-the-loop benchmark:** we deliberately exclude model stochasticity to isolate control flow, so we report no task-success or cost metrics; the LangChain result [6] is cited as external evidence, not our measurement. **Generality:** the claim $L \subsetneq H$ rests on the "everything but the model" definition of harness.

11. Conclusion and Future Work

Loop engineering and harness engineering are a part and a whole. The harness is the spatial scaffolding around a model; the loop is the temporal control structure that is one component of it. Loop engineering's distinctive contribution is to make verification and termination first-class, and the most useful consequence is to enforce them as type-level obligations, as OnLOOP does, rather than as discretionary practice. The bare Ralph loop is precisely the degenerate case that omits both. Future work: (1) a model-in-the-loop evaluation with LLM-backed actors and real sensors against single-prompt and bare-Ralph baselines; (2) implementation of the outer steering loop as an automated guide-writer (a `GuideStore` contract); (3) isolation and safety mechanisms (worktrees, human-on-the-loop checkpoints) as additional invariants.

References

[1] A. Osmani. *Loop Engineering: Designing Systems That Prompt AI Agents*, 2026. (Popularization, building on P. Steinberger and B. Cherny.) https://lushbinary.com/blog/loop-engineering-ai-coding-agents-guide/

[2] A. Osmani. *Agent Harness Engineering*, 2026. https://addyosmani.com/blog/agent-harness-engineering/

[3] B. Böckeler. *Harness Engineering for Coding Agent Users*. martinfowler.com, 2026. https://martinfowler.com/articles/harness-engineering.html

[4] G. Huntley (Ralph loop / Ralph Wiggum technique); see T. Wiegold, *The Ralph Loop: How Recursive AI Agents Actually Work*, 2026. https://thomas-wiegold.com/blog/ralph-loop-how-recursive-ai-agents-work/

[5] S. Bharath. *Ralph Wiggum: The Dumbest Smart Way to Run Coding Agents*, 2026. https://sidbharath.com/blog/ralph-wiggum-claude-code/

[6] LangChain. *The Anatomy of an Agent Harness*, 2026. https://www.langchain.com/blog/the-anatomy-of-an-agent-harness

[7] Oracle. *What Is the AI Agent Loop?*, 2026. https://blogs.oracle.com/developers/what-is-the-ai-agent-loop-the-core-architecture-behind-autonomous-ai-systems

[8] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 1995.

[9] Thoughtworks. *Harness Engineering and Agent Feedback: Exploring AI Coding Sensors*, 2026.

Table 1: The loop modeled as a six-tuple $L = (G, W, A, S, M_s, T)$.

Symbol	Role	Signature (informal)
G	Goal	`is_satisfied : State → Bool`
W	WorkSource	`find : State → List[WorkItem]`
A	Actor	`act : WorkItem × Context → ActionResult`
S	Sensor(s)	`verify : WorkItem × ActionResult × Context → Verdict`
M_s	Memory / StateStore	`load / save / record` over a persistent `State`
T	StopCondition(s)	`should_stop : State → Decision`

```
repeat:
    if exists t in T . should_stop(s) then halt(reason)      # (R1) stop is checked first
    items <- { w in W.find(s) | id(w) not in done(s) }
    if items = empty then s <- bump_empty(s); continue       # dryness accrues
    w <- head(items)
    r <- A.act(w, s)
    v <- AND over s' in S of s'.verify(w, r, s)              # (R2) conjunction of sensors
    if passed(v) then s <- s[ done <- done + {id(w)} ]       # (R3) progress iff verified
    s <- M_s.record(s, <w, r, v, tau>); persist(s)          # (R4) durable, restartable
```

Figure 4: Execution semantics of an engineered loop. State σ written `s`; timestamp τ written `tau`. Rules R1–R4 are referenced in the text.

```
Agent
+- Harness            <- harness engineering
   |  designs all of this
   +- Guides (feedforward): conventions,
   |     docs, templates, linters
   +- Sensors (feedback): tests, static
   |     analysis, review agents
   +- Tools / Memory / Sandbox
   +- Loop (L)         <- loop engineering
         designs this part
         G -> W.find -> A.act ->
         S.verify -> M_s -> T -> repeat
```

Figure 5: The part–whole relation. The harness is the spatial scaffolding; the loop is the temporal control structure that is one component of it.

Table 2: Loop engineering vs. harness engineering across ten dimensions.

Dimension	Loop Engineering	Harness Engineering
Primary object	The self-running cycle that drives the agent	The full scaffolding around the model
Axis	Temporal / control-flow (when, how often, until when)	Spatial / structural (what surrounds the model)
Unit of leverage	The system that generates and verifies prompts	The constraints and feedback that shape output
Input $\rightarrow$ output	Goal $+$ model oracle $+$ workspace $+$ prior state $\rightarrow$ stop reason $+$ verified done-set $+$ mutated workspace $+$ replayable trace	Model $+$ task context (guides, tools, memory) $\rightarrow$ one constrained, sensor-checked action
Failure mode	Human is the bottleneck; cannot run unattended	Output is unreliable; agent does the wrong thing
Control-theory analogue	The closed-loop controller and its stopping rule	The full plant: actuators, sensors, setpoints
Verification stance	Verification is a stage of the cycle (S)	Verification is one of two control populations
Termination	First-class concern (T); core to the discipline	Implicit; not central
Lifecycle / ownership	Often automated, scheduled, recursive	Iteratively improved by a human meta-loop
Canonical artifact	A loop spec; the executor	Guides + sensors + tools + sandboxes